

SECTION 6 EXCEPTION PROCESSING

Exception processing is defined as the activities performed by the processor in preparing to execute a handler routine for any condition that causes an exception. In particular, exception processing does not include execution of the handler routine itself. An introduction to exception processing, as one of the processing states of the MC68020/EC020, is given in **Section 2 Processing States**.

This section describes exception processing in detail, describing the processing for each type of exception. It describes the return from an exception and bus fault recovery. This section also describes the formats of the exception stack frames. For more detail on protocol violation and coprocessor-related exceptions, refer to **Section 7 Coprocessor Interface Description**. Also, for more detail on exceptions defined for floating-point coprocessors, refer to MC68881UM/AD, *MC68881/MC68882 Floating-Point Coprocessor User's Manual*.

6.1 EXCEPTION PROCESSING SEQUENCE

Exception processing occurs in four functional steps. However, all individual bus cycles associated with exception processing (vector acquisition, stacking, etc.) are not guaranteed to occur in the order in which they are described in this section. Nonetheless, all addresses and offsets from the stack pointer are guaranteed to be as described.

The first step of exception processing involves the SR. The processor makes an internal copy of the SR, then sets the S-bit in the SR, changing to the supervisor privilege level. Next, the processor inhibits tracing of the exception handler by clearing the T1 and T0 bits in the SR. For the reset and interrupt exceptions, the processor also updates the interrupt priority mask (bits 10–8 of the SR).

In the second step, the processor determines the vector number of the exception. For interrupts, the processor performs an interrupt acknowledge cycle (a read from the CPU address space type 1111; see Figures 5-32 and 5-33) to obtain the vector number. For coprocessor-detected exceptions, the vector number is included in the coprocessor exception primitive response. (Refer to **Section 7 Coprocessor Interface Description** for a complete discussion of coprocessor exceptions.) For all other exceptions, internal logic provides the vector number. This vector number is used in the last step to calculate the address of the exception vector. Throughout this section, vector numbers are given in decimal notation.

For all exceptions other than reset, the third step is to save the current processor context. The processor creates an exception stack frame on the active supervisor stack and fills it with context information appropriate for the type of exception. Other information may also be stacked, depending on which exception is being processed and the state of the processor prior to the exception. If the exception is an interrupt and the M-bit in the SR is set, the processor clears the M-bit and builds a second stack frame on the interrupt stack.

The last step initiates execution of the exception handler. The processor multiplies the vector number by four to determine the exception vector offset. The processor then adds the offset to the value stored in the VBR to obtain the memory address of the exception vector. Next, the processor loads the PC (and the ISP for the reset exception) from the exception vector table in memory. After prefetching the first three words to fill the instruction pipe, the processor resumes normal processing at the address in the PC. Table 6-1 contains a description of all the exception vector offsets defined for the MC68020/EC020.

As shown in Table 6-1, the first 64 vectors are defined by Motorola, and 192 vectors are reserved for interrupt vectors defined by the user. However, external devices may use vectors reserved for internal purposes at the discretion of the system designer.

Table 6-1. Exception Vector Assignments

Vector Number	Vector Offset		Assignment
	Hex	Space	
0	000	SP	Reset Initial Interrupt Stack Pointer
1	004	SP	Reset Initial Program Counter
2	008	SD	Bus Error
3	00C	SD	Address Error
4	010	SD	Illegal Instruction
5	014	SD	Zero Divide
6	018	SD	CHK, CHK2 Instruction
7	01C	SD	cpTRAPcc, TRAPcc, TRAPV Instructions
8	020	SD	Privilege Violation
9	024	SD	Trace
10	028	SD	Line 1010 Emulator
11	02C	SD	Line 1111 Emulator
12	030	SD	(Unassigned, Reserved)
13	034	SD	Coprocessor Protocol Violation
14	038	SD	Format Error
15	03C	SD	Uninitialized Interrupt
16–23	040	SD	Unassigned, Reserved
	05C	SD	
24	060	SD	Spurious Interrupt
25	064	SD	Level 1 Interrupt Autovector
26	068	SD	Level 2 Interrupt Autovector
27	06C	SD	Level 3 Interrupt Autovector
28	070	SD	Level 4 Interrupt Autovector
29	074	SD	Level 5 Interrupt Autovector
30	078	SD	Level 6 Interrupt Autovector
31	07C	SD	Level 7 Interrupt Autovector
32–47	080	SD	TRAP #0–15 Instruction Vectors
	0BC	SD	
48	0C0	SD	FPCP Branch or Set on Unordered Condition
49	0C4	SD	FPCP Inexact Result
50	0C8	SD	FPCP Divide by Zero
51	0CC	SD	FPCP Underflow
52	0D0	SD	FPCP Operand Error
53	0D4	SD	FPCP Overflow
54	0D8	SD	FPCP Signaling NAN
55	0DC	SD	Unassigned, Reserved
56	0E0	SD	PMMU Configuration
57	0E4	SD	PMMU Illegal Operation
58	0E8	SD	PMMU Access Level Violation
59–63	0EC	SD	Unassigned, Reserved
	0FC	SD	
64–255	100	SD	User-Defined Vectors (192)
	3FC	SD	

SP—Supervisor Program Space

SD—Supervisor Data Space

6.1.1 Reset Exception

Assertion of the RESET signal by external hardware causes a reset exception. For details on the requirements for the assertion of RESET, refer to **Section 5 Bus Operation**.

The reset exception has the highest priority of any exception; it provides for system initialization and recovery from catastrophic failure. When a reset exception is recognized, it aborts any processing in progress and that processing cannot be recovered. Figure 6-1 is a flowchart of the reset exception, which performs the following operations:

1. Clears the T1 and T0 bits in the SR to disable tracing.
2. Places the processor in the interrupt mode of the supervisor privilege level by setting the S-bit and clearing the M-bit in the SR.
3. Sets the I2–I0 bits in the SR to the highest priority level (level 7).
4. Initializes the VBR to zero (\$00000000).
5. Clears the E and F bits in the CACR.
6. Invalidates all entries in the instruction cache.
7. Generates a vector number to reference the reset exception vector (two long words) at offset zero in the supervisor program address space.
8. Loads the first long word of the reset exception vector into the interrupt stack pointer.
9. Loads the second long word of the reset exception vector into the PC.

After the initial instruction prefetches, program execution begins at the address in the PC. The reset exception does not save the value of either the PC or the SR.

As described in **Section 5 Bus Operation**, if a bus error or address error occurs during the exception processing sequence for a reset, a double bus fault occurs. The processor halts and asserts the HALT signal to indicate the halted condition.

Execution of the RESET instruction does not cause a reset exception, nor does it affect any internal registers, but it does cause the MC68020/EC020 to assert the RESET signal, resetting all external devices.

6.1.2 Bus Error Exception

A bus error exception occurs when external logic aborts a bus cycle by asserting the BERR signal. If the aborted bus cycle is a data access, the processor immediately begins exception processing. If the aborted bus cycle is an instruction prefetch, the processor may delay taking the exception until it attempts to use the prefetched information.

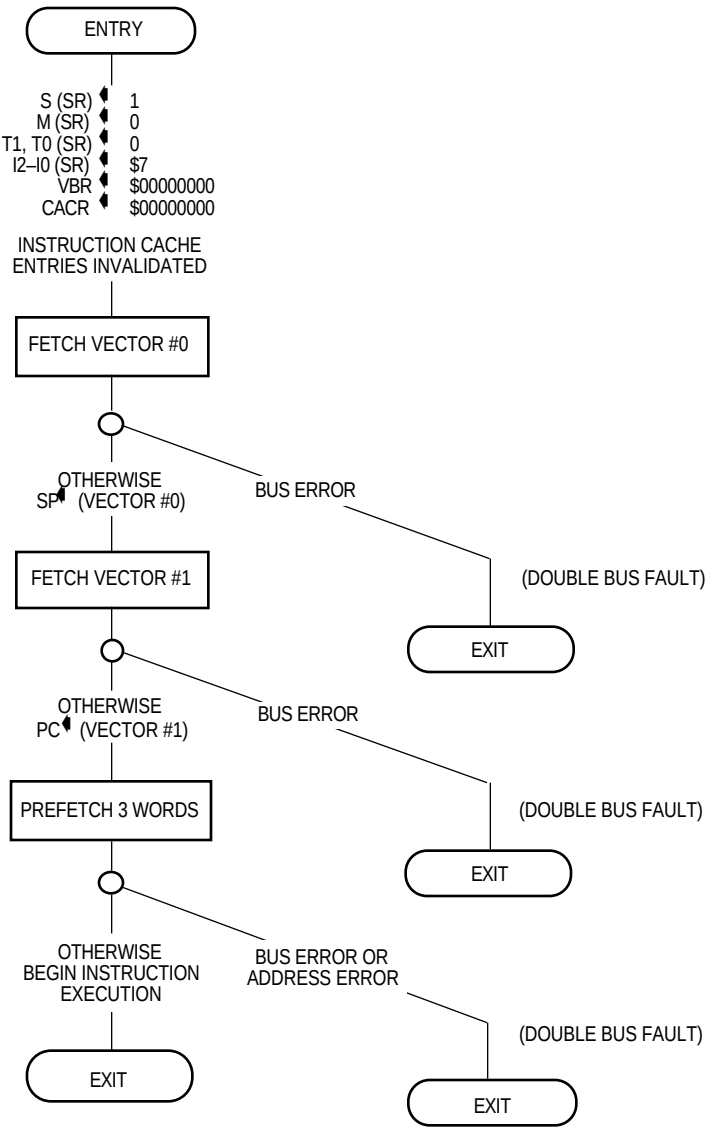


Figure 6-1. Reset Operation Flowchart

The processor begins exception processing for a bus error by making an internal copy of the current SR. The processor then enters the supervisor privilege level (by setting the S-bit in the SR) and clears the T1 and T0 bits in the SR. The processor generates exception vector number 2 for the bus error vector. It saves the vector offset, PC, and the internal copy of the SR on the active supervisor stack. The saved PC value is the logical address of the instruction that was executing at the time the fault was detected. This is not necessarily the instruction that initiated the bus cycle since the processor overlaps

execution of instructions. The processor also saves the contents of some of its internal registers. The information saved on the stack is sufficient to identify the cause of the bus fault and recover from the error.

For efficiency, the MC68020/EC020 uses two different bus error stack frame formats. When the bus error exception is taken at an instruction boundary, less information is required to recover from the error, and the processor builds the short bus fault stack frame as shown in Table 6-5. When the exception is taken during the execution of an instruction, the processor must save its entire state for recovery and uses the long bus fault stack frame shown in Table 6-5. The format code in the stack frame distinguishes the two stack frame formats. Stack frame formats are described in detail in **6.4 Exception Stack Frame Formats**.

If a bus error occurs during the exception processing for a bus error, address error, or reset or while the processor is loading internal state information from the stack during the execution of an RTE instruction, a double bus fault occurs and the processor enters the halted state. In this case, the processor does not attempt to alter the current state of memory. Only an external RESET can restart a processor halted by a double bus fault.

6.1.3 Address Error Exception

An address error exception occurs when the processor attempts to prefetch an instruction from an odd address. This exception is similar to a bus error exception but is internally initiated. A bus cycle is not executed, and the processor begins exception processing immediately. After exception processing commences, the sequence is the same as that for bus error exceptions described in the preceding paragraphs, except that the vector number is 3 and the vector offset in the stack frame refers to the address error vector. Either a short or long bus fault stack frame may be generated. If an address error occurs during the exception processing for a bus error, address error, or reset, a double bus fault occurs.

6.1.4 Instruction Trap Exception

Certain instructions are used to explicitly cause trap exceptions. The TRAP instruction always forces an exception and is useful for implementing system calls in user programs. The TRAPcc, TRAPV, cpTRAPcc, CHK, and CHK2 instructions force exceptions if the user program detects an error, which may be an arithmetic overflow or a subscript value that is out of bounds.

The DIVS and DIVU instructions force exceptions if a division operation is attempted with a divisor of zero.

When a trap exception occurs, the processor copies the SR internally, enters the supervisor privilege level (by setting the S-bit in the SR), and clears the T1 and T0 bits in the SR. If tracing is enabled for the instruction that caused the trap, a trace exception is taken after the RTE instruction from the trap handler is executed, and the trace corresponds to the trap instruction; the trap handler routine is not traced. The processor generates a vector number according to the instruction being executed; for the TRAP

instruction, the vector number is 32 plus n. The stack frame saves the trap vector offset, the PC, and the internal copy of the SR on the supervisor stack. The saved value of the PC is the logical address of the instruction following the instruction that caused the trap. For all instruction traps other than TRAP, a pointer to the instruction that caused the trap is also saved. Instruction execution resumes at the address in the exception vector after the required instruction prefetches.

6.1.5 Illegal Instruction and Unimplemented Instruction Exceptions

An illegal instruction is an instruction that contains any bit pattern in its first word that does not correspond to the bit pattern of the first word of a valid MC68020/EC020 instruction or a MOVEC instruction with an undefined register specification field in the first extension word. An illegal instruction exception corresponds to vector number 4 and occurs when the processor attempts to execute an illegal instruction.

An illegal instruction exception is also taken if a breakpoint acknowledge bus cycle (see **Section 5 Bus Operation**) is terminated with the assertion of the BERR signal. This implies that the external circuitry did not supply an instruction word to replace the BKPT instruction word in the instruction pipe.

Instruction word patterns with bits 15–12 = 1010 are referred to as unimplemented instructions with A-line opcodes. When the processor attempts to execute an unimplemented instruction with an A-line opcode, an exception is generated with vector number 10, permitting efficient emulation of unimplemented instructions.

Instructions that have word patterns with bits 15–12 = 1111, bits 11–9 = 000, and defined word patterns for subsequent words, are legal PMMU instructions. Instructions that have bits 15–12 of the first words = 1111, bits 11–9 = 000, and undefined patterns in the subsequent words, are treated as unimplemented instructions with F-line opcodes when execution is attempted in the supervisor privilege level. When execution of the same instruction is attempted in the user privilege level, a privilege violation exception is taken. The exception vector number for an unimplemented instruction with an F-line opcode is 11.

The word patterns with bits 15–12 = 1111 and bits 11–9 $\neq$ 000 are used for coprocessor instructions. When the processor identifies a coprocessor instruction, it runs a bus cycle referencing CPU space type \$2 (refer to **Section 2 Processing States**) and addressing one of eight coprocessors (0–7, according to bits 11–9). If the addressed coprocessor is not included in the system and the cycle terminates with the assertion of BERR, the instruction takes an unimplemented instruction (F-line opcode) exception. The system can emulate the functions of the coprocessor with an F-line exception handler. Refer to **Section 7 Coprocessor Interface Description** for more details.

Exception processing for illegal and unimplemented instructions is similar to that for instruction traps. When the processor has identified an illegal or unimplemented instruction, it initiates exception processing instead of attempting to execute the instruction. The processor copies the SR, enters the supervisor privilege level (by setting the S bit in the SR), and clears the T1 and T0 bits in the SR, disabling further tracing. The processor generates the vector number, either 4, 10, or 11, according to the exception type. The illegal or unimplemented instruction vector offset, current PC, and copy of the SR are saved on the supervisor stack, with the saved value of the PC being the address of the illegal or unimplemented instruction. Instruction execution resumes at the address contained in the exception vector. It is the responsibility of the handling routine to adjust the stacked PC if the instruction is emulated in software or is to be skipped on return from the handler.

6.1.6 Privilege Violation Exception

To provide system security, the following instructions are privileged:

- ANDI to SR
- EORI to SR
- cpRESTORE
- cpSAVE
- MOVE from SR
- MOVE to SR
- MOVE USP
- MOVEC
- MOVES
- ORI to SR
- RESET
- RTE
- STOP

An attempt to execute one of the privileged instructions while at the user privilege level causes a privilege violation exception. Also, a privilege violation exception occurs if a coprocessor requests a privilege check and the processor is at the user level.

Exception processing for privilege violations is similar to that for illegal instructions. When the processor identifies a privilege violation, it begins exception processing before executing the instruction. The processor copies the SR, enters the supervisor privilege level by setting the S-bit in the SR, and clears the T1 and T0 bits in the SR. The processor generates vector number 8, the privilege violation exception vector, and saves the privilege violation vector offset, the current PC value, and the internal copy of the SR on the supervisor stack. The saved value of the PC is the logical address of the first word of the instruction that caused the privilege violation. Instruction execution resumes after the required prefetches from the address in the privilege violation exception vector.

6.1.7 Trace Exception

To aid in program development, the M68000 processors include an instruction-by-instruction tracing capability. The MC68020/EC020 can be programmed to trace all instructions or only instructions that change program flow. In the trace mode, an instruction generates a trace exception after it completes execution, allowing a debugger program to monitor execution of a program.

The T1 and T0 bits in the supervisor portion of the SR control tracing. The state of these bits when an instruction begins execution determines whether the instruction generates a trace exception after the instruction completes. Clearing both the T1 and T0 bits disables tracing, and instruction execution proceeds normally. Clearing the T1 bit and setting the T0 bit causes an instruction that forces a change of flow to take a trace exception. Instructions that increment the PC normally do not take the trace exception. Instructions that are traced in this mode include all branches, jumps, instruction traps, returns, and coprocessor instructions that modify the PC flow. This mode also includes SR manipulations because the processor must re-prefetch instruction words to fill the pipe again any time an instruction that can modify the SR is executed. The execution of the BKPT instruction causes a change of flow if the opcode replacing the BKPT is an instruction that causes a change of flow (i.e., a jump, branch, etc.). Setting the T1 bit and clearing the T0 bit causes the execution of all instructions to force trace exceptions. Table 6-2 shows the trace mode selected by each combination of T1 and T0.

Table 6-2. Tracing Control

T1	T0	Tracing Function
0	0	No Tracing
0	1	Trace on Change of Flow (BRA, JMP, etc.)
1	0	Trace on Instruction Execution (Any Instruction)
1	1	Undefined, Reserved

In general terms, a trace exception is an extension to the function of any traced instruction—i.e., the execution of a traced instruction is not complete until completion of trace exception processing. If an instruction does not complete due to a bus error or address error exception, trace exception processing is deferred until after the execution of the suspended instruction is resumed, and the instruction execution completes normally. If an interrupt is pending at the completion of an instruction, the trace exception processing occurs before the interrupt exception processing starts. If an instruction forces an exception as part of its normal execution, the forced exception processing occurs before the trace exception is processed. See **6.1.11 Multiple Exceptions** for a more complete discussion of exception priorities.

When tracing is enabled and the processor attempts to execute an illegal or unimplemented instruction, that instruction does not cause a trace exception since it is not executed. This is of particular importance to an instruction emulation routine that performs the instruction function, adjusts the stacked PC to skip the unimplemented instruction, and returns. Before returning, the T1 and T0 bits of the SR on the stack should be checked. If

tracing is enabled, the trace exception processing should also be emulated for the trace exception handler to account for the emulated instruction.

The exception processing for a trace starts at the end of normal processing for the traced instruction and before the start of the next instruction. The processor makes an internal copy of the SR and enters the supervisor privilege level (by setting the S-bit in the SR). It also clears the T0 and T1 bits of the SR, disabling further tracing. The processor supplies vector number 9 for the trace exception and saves the trace exception vector offset, PC value, and the copy of the SR on the supervisor stack. The saved value of the PC is the logical address of the next instruction to be executed. Instruction execution resumes after the required prefetches from the address in the trace exception vector.

The STOP instruction does not perform its function when it is traced. A STOP instruction that begins execution with T1, T0 = 10 forces a trace exception after it loads the SR. Upon return from the trace handler routine, execution continues with the instruction following the STOP instruction, and the processor never enters the stopped condition.

6.1.8 Format Error Exception

Just as the processor checks that prefetched instructions are valid, the processor (with the aid of a coprocessor, if needed) also performs some checks of data values for control operations, including the type and option fields of the descriptor for CALLM, the coprocessor state frame format word for a cpRESTORE instruction, and the stack frame format for an RTE or an RTM instruction.

The RTE instruction checks the validity of the stack format code. For long bus fault format frames, the RTE instruction also compares the internal version number of the processor to that contained in the frame at memory location SP + 54 (SP + \$36). This check ensures that the processor can correctly interpret internal state information from the stack frame.

The CALLM and RTM both check the values in the option and type fields in the module descriptor and module stack frame, respectively. If these fields do not contain proper values or if an illegal access rights change request is detected by an external memory management unit, then an illegal call or return is being requested and is not executed. Refer to **Section 9 Applications Information** for more information on the module call/return mechanism.

The cpRESTORE instruction passes the format word of the coprocessor state frame to the coprocessor for validation. If the coprocessor does not recognize the format value, it signals the MC68020/EC020 to take a format error exception. Refer to **Section 7 Coprocessor Interface Description** for details of coprocessor-related exceptions.

If any of the checks previously described determine that the format of the stacked data is improper, the instruction generates a format error exception. This exception saves a short bus fault stack frame, generates exception vector number 14, and continues execution at the address in the format exception vector. The stacked PC value is the logical address of the instruction that detected the format error.

6.1.9 Interrupt Exceptions

When a peripheral device requires the services of the MC68020/EC020 or is ready to send information that the processor requires, it may signal the processor to take an interrupt exception. The interrupt exception transfers control to a routine that responds appropriately.

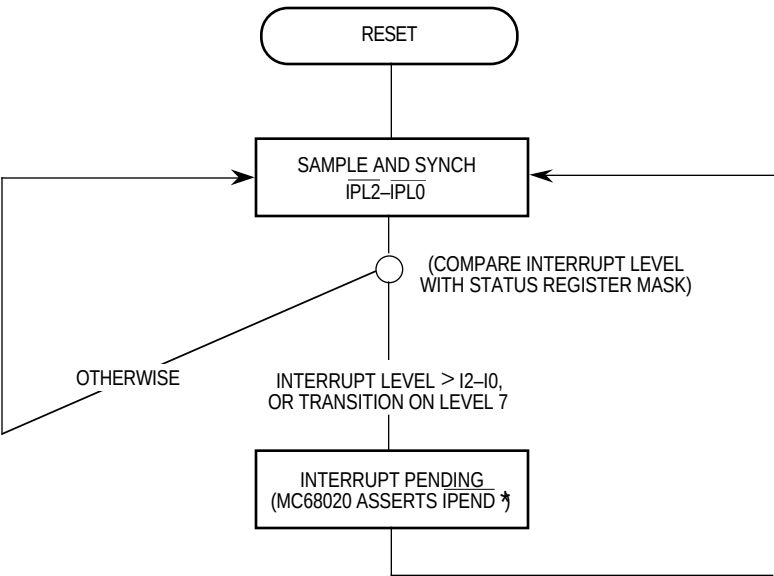
The peripheral device uses the IPL2–IPL0 signals to signal an interrupt condition to the processor and to specify the priority of that condition. These three signals encode a value of zero through seven (IPL0 is the least significant bit). When IPL2–IPL0 are all negated, the interrupt request level is zero. IPL2–IPL0 values 1–7 specify one of seven levels of prioritized interrupts; level 7 has the highest priority. External circuitry can chain or otherwise merge signals from devices at each level, allowing an unlimited number of devices to interrupt the processor.

The IPL2–IPL0 signals must maintain the interrupt request level until the MC68020/EC020 acknowledges the interrupt to guarantee that the interrupt is recognized. The MC68020/EC020 continuously samples the IPL2–IPL0 signals on consecutive falling edges of the processor clock to synchronize and debounce these signals. An interrupt request that is the same for two consecutive falling clock edges is considered a valid input. Although the protocol requires that the request remain until the processor runs an interrupt acknowledge cycle for that interrupt value, an interrupt request that is held for as short a period as two clock cycles could be recognized.

The I2–I0 bits in the SR specify the interrupt priority mask. The value in the interrupt mask is the highest priority level that the processor ignores. When an interrupt request has a priority higher than the value in the mask, the processor makes the request a pending interrupt. Figure 6-2 is a flowchart of the procedure for making an interrupt pending.

When several devices are connected to the same interrupt level, each device should hold its interrupt priority level constant until its corresponding interrupt acknowledge cycle to ensure that all requests are processed.

Table 6-3 lists the interrupt levels, the states of IPL2–IPL0 that define each level, and the mask value that allows an interrupt at each level.



*IPEND is not implemented in the MC68EC020.

Figure 6-2. Interrupt Pending Procedure

Table 6-3. Interrupt Levels and Mask Values

Requested Interrupt Level	Control Line Status			Interrupt Mask Value Required for Recognition
	IPL2	IPL1	IPL0	
0*	N	N	N	N/A*
1	N	N	A	0
2	N	A	N	1-0
3	N	A	A	2-0
4	A	N	N	3-0
5	A	N	A	4-0
6	A	A	N	5-0
7	A	A	A	7-0

*Indicates that no interrupt is requested.

A—Asserted

N—Negated

Priority level 7, the nonmaskable interrupt, is a special case. Level 7 interrupts cannot be masked by the interrupt priority mask, and they are transition sensitive. The processor recognizes an interrupt request each time the external interrupt request level changes from some lower level to level 7, regardless of the value in the mask. Figure 6-3 shows two examples of interrupt recognitions, one for level 6 and one for level 7. When the MC68020/EC020 processes a level 6 interrupt, the interrupt priority mask is automatically updated with a value of 6 before entering the handler routine so that subsequent level 6 interrupts are masked. Provided no instruction that lowers the mask value is executed, the

external request can be lowered to level 3 and then raised back to level 6, and a second

level 6 interrupt is not processed. However, if the MC68020/EC020 is handling a level 7 interrupt (I2–I0 in the SR set to 111) and the external request is lowered to level 3 and then raised back to level 7, a second level 7 interrupt is processed. The second level 7 interrupt is processed because the level 7 interrupt is transition sensitive. A level 7 interrupt is also generated by a level comparison if the request level and mask level are at 7 and the priority mask is then set to a lower level (with the MOVE to SR or RTE instruction, for example). As shown in Figure 6-3 for level 6 interrupt request level and mask level, this is the case for all interrupt levels.

Note that a mask value of 6 and a mask value of 7 both inhibit request levels of 1–6 from being recognized. In addition, neither masks a transition to an interrupt request level of 7. The only difference between mask values of 6 and 7 occurs when the interrupt request level is 7 and the mask value is 7. If the mask value is lowered to 6, a second level 7 interrupt is recognized.

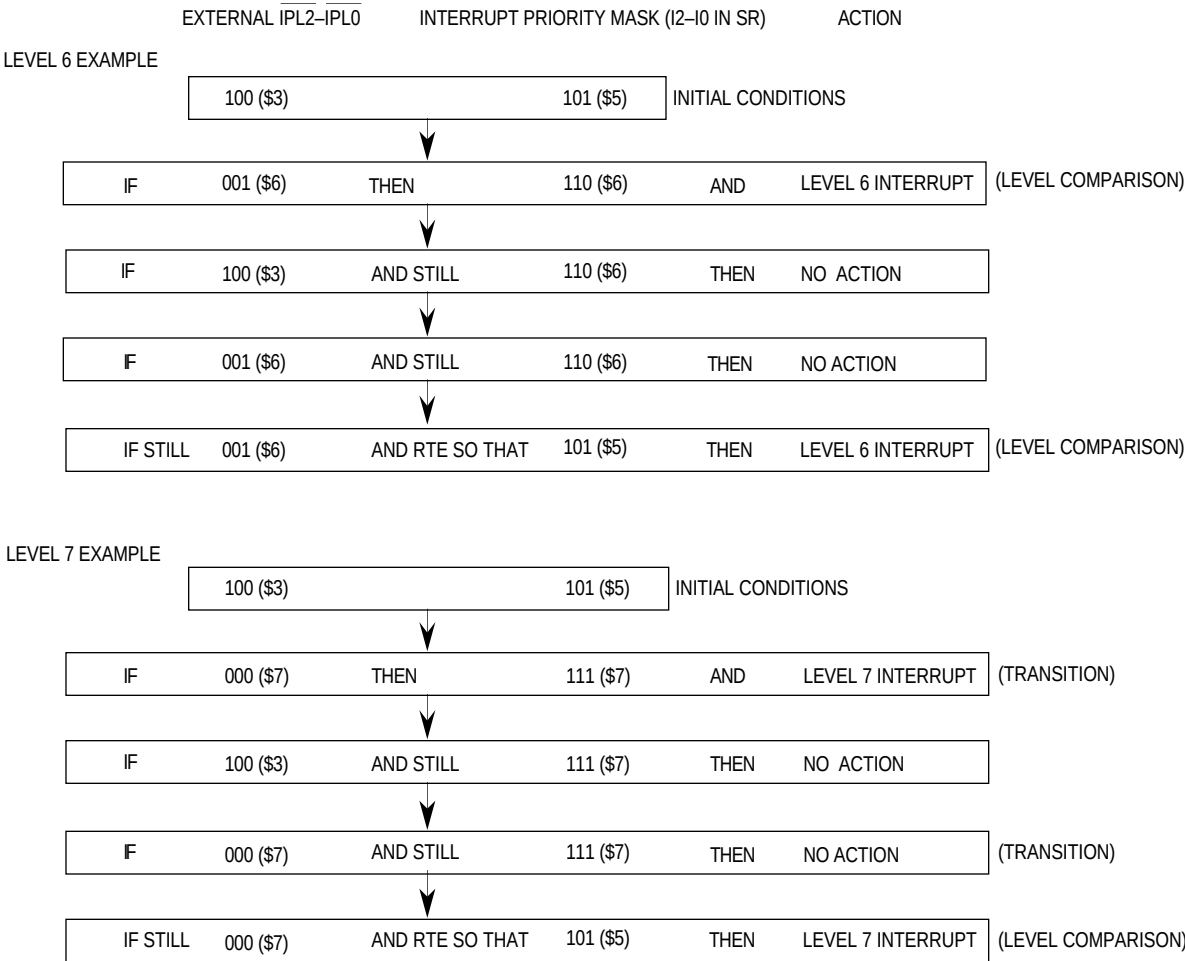


Figure 6-3. Interrupt Recognition Examples

The MC68020 asserts IPEND (note that IPEND is not implemented in the MC68EC020) when it makes an interrupt request pending. Figure 6-4 shows the assertion of IPEND relative to the assertion of an interrupt level on IPL2–IPL0. IPEND signals to external devices that an interrupt exception will be taken at an upcoming instruction boundary (following any higher priority exception). The state of the IPEND signal is internally checked by the processor once per instruction, independently of bus operation. In addition, it is checked during the second instruction prefetch associated with exception processing.

Figure 6-5 is a flowchart of the interrupt recognition and associated exception processing sequence.

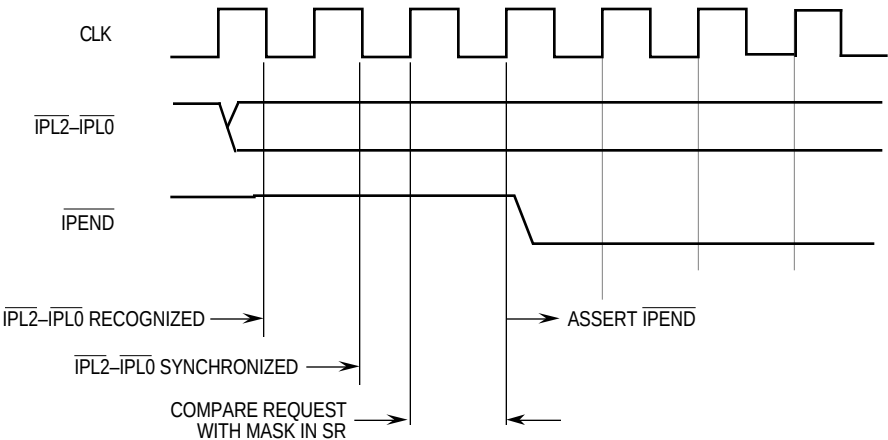
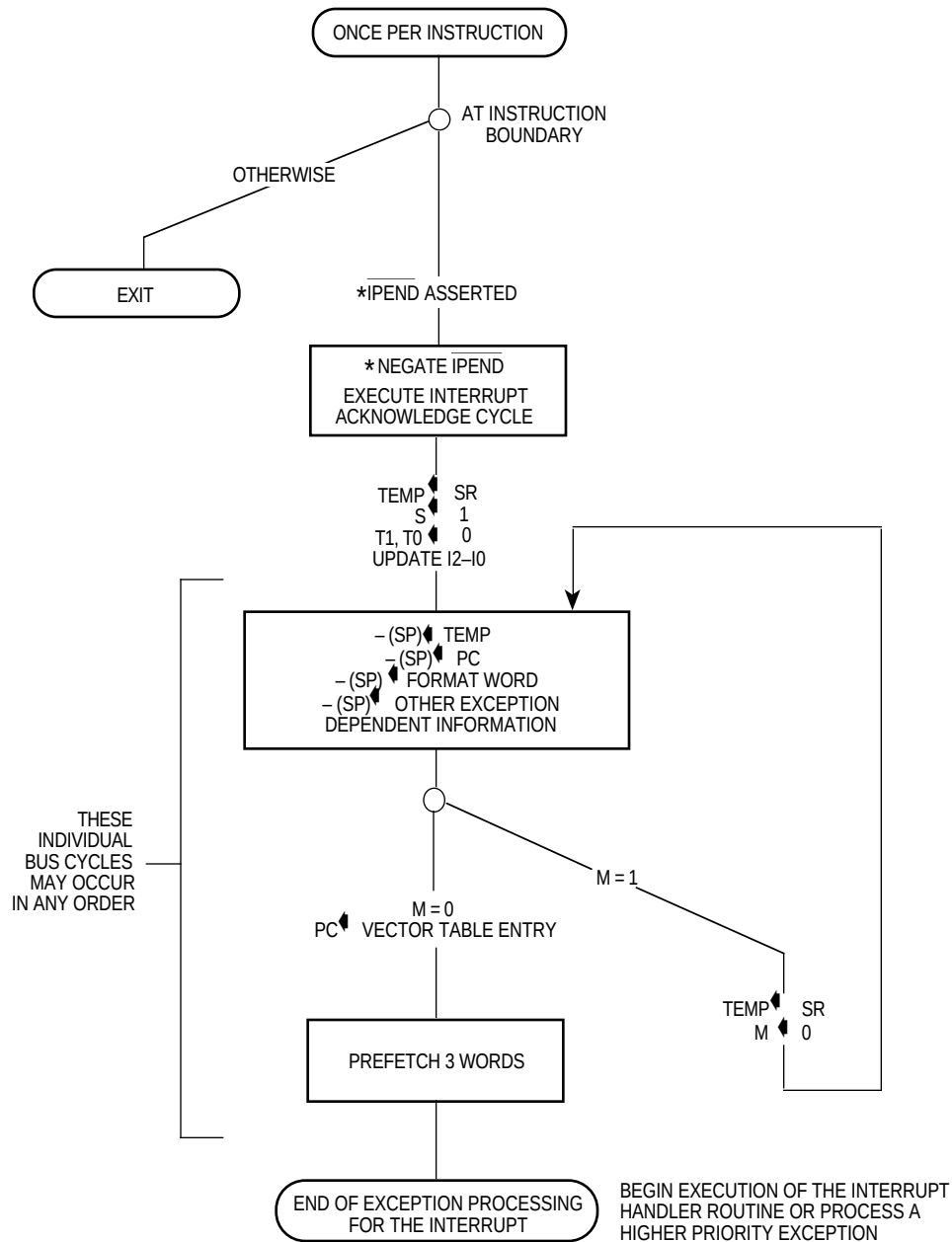


Figure 6-4. Assertion of IPEND (MC68020 Only)



* Does not apply to the MC68EC020.

Figure 6-5. Interrupt Exception Processing Flowchart

For the MC68020, if no higher priority interrupt has been synchronized, the IPEND signal is negated during state 0 (S0) of an interrupt acknowledge cycle, and the IPL2–IPL0 signals for the interrupt being acknowledged can be negated at this time. For the MC68EC020, if no higher priority interrupt has been synchronized, the IPL2–IPL0 signals for the interrupt being acknowledged can be negated at this time. Refer to **Section 5 Bus Operation** for more information on interrupt acknowledge cycles.

When processing an interrupt exception, the MC68020/EC020 first makes an internal copy of the SR, sets the privilege level to supervisor, suppresses tracing, and sets the processor interrupt mask level to the level of the interrupt being serviced. The processor attempts to obtain a vector number from the interrupting device using an interrupt acknowledge bus cycle with the interrupt level number output on pins A3–A1 of the address bus. For a device that cannot supply an interrupt vector, the AVEC signal can be asserted, and the MC68020/EC020 uses an internally generated autovector, which is one of vector numbers 31–25, that corresponds to the interrupt level number. If external logic indicates a bus error during the interrupt acknowledge cycle, the interrupt is considered spurious, and the processor generates the spurious interrupt vector number (24). Refer to **Section 5 Bus Operation** for complete interrupt bus cycle information.

Once the vector number is obtained, the processor saves the exception vector offset, PC value, and the internal copy of the SR on the active supervisor stack. The saved value of the PC is the logical address of the instruction that would have been executed had the interrupt not occurred. If the interrupt was acknowledged during the execution of a coprocessor instruction, further internal information is saved on the stack so that the MC68020/EC020 can continue executing the coprocessor instruction when the interrupt handler completes execution.

If the M-bit in the SR is set, the processor clears the M-bit and creates a throwaway exception stack frame on top of the interrupt stack as part of interrupt exception processing. This second frame contains the same PC value and vector offset as the frame created on top of the master stack, but has a format number of 1 instead of 0 or 9. The copy of the SR saved on the throwaway frame is exactly the same as that placed on the master stack except that the S-bit is set in the version placed on the interrupt stack. (It may or may not be set in the copy saved on the master stack.) The resulting SR (after exception processing) has the S-bit set and the M-bit cleared.

The processor loads the address in the exception vector into the PC, and normal instruction execution resumes after the required prefetches for the interrupt handler routine.

Most M68000 family peripherals use programmable interrupt vector numbers as part of the interrupt request/acknowledge mechanism of the system. If this vector number is not initialized after reset and the peripheral must acknowledge an interrupt request, the peripheral usually returns the uninitialized interrupt vector number (15).

6.1.10 Breakpoint Instruction Exception

To use the MC68020/EC020 in a hardware emulator, it must provide a means of inserting breakpoints in the emulator code and of performing appropriate operations at each breakpoint. For the MC68000 and MC68008, this can be done by inserting an illegal instruction at the breakpoint and detecting the illegal instruction exception from its vector location. However, since the VBR on M68000 family processors MC68010 and later allows arbitrary relocation of exception vectors, the exception address cannot reliably identify a breakpoint. The MC68020/EC020 processor provides a breakpoint capability with a set of breakpoint instructions, \$4848–\$484F, for eight unique breakpoints. The breakpoint facility also allows external hardware to monitor the execution of a program residing in the on-chip instruction cache without severe performance degradation.

When the MC68020/EC020 executes a breakpoint instruction, it performs a breakpoint acknowledge cycle (read cycle) from CPU space type \$0 with address lines A4–A2 corresponding to the breakpoint number. Refer to **Section 5 Bus Operation** for a description of the breakpoint acknowledge cycle. The external hardware can return either BERR or DSACK1/DSACK0 with an instruction word on the data bus. If the bus cycle terminates with BERR, the processor performs illegal instruction exception processing. If the bus cycle terminates with DSACK1/DSACK0, the processor uses the data returned to replace the breakpoint instruction in the internal instruction pipe and begins execution of that instruction. The remainder of the pipe remains unaltered. In addition, no stacking or vector fetching is involved with the execution of the instruction. Figure 6-6 is a flowchart of the breakpoint instruction execution.

6.1.11 Multiple Exceptions

When several exceptions occur simultaneously, they are processed according to a fixed priority. Table 6-4 lists the exceptions grouped by characteristics. Each group has a priority from 4–0. Priority 0 has the highest priority.

As soon as the MC68020/EC020 has completed exception processing for a condition when another exception is pending, it begins exception processing for the pending exception instead of executing the exception handler for the original exception condition. Also, whenever a bus error or address error occurs, its exception processing takes precedence over lower priority exceptions and occurs immediately. For example, if a bus error occurs during the exception processing for a trace condition, the system processes the bus error and executes its handler before completing the trace exception processing. However, most exceptions cannot occur during exception processing, and very few combinations of the exceptions shown in Table 6-4 can be pending simultaneously.

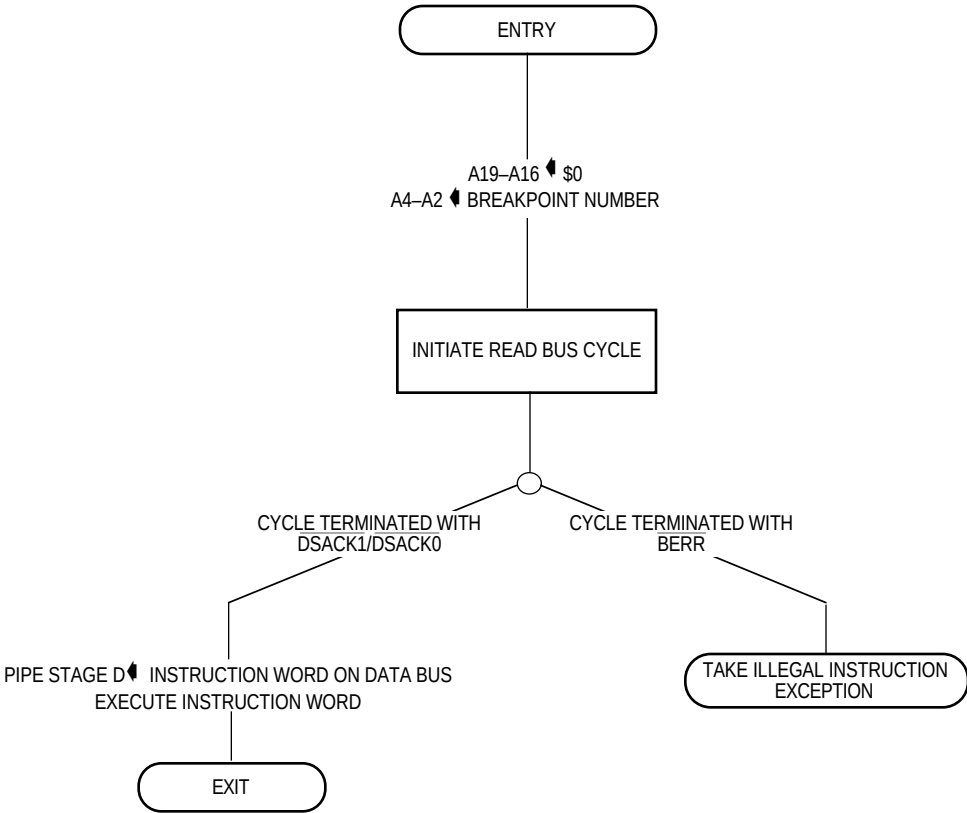


Figure 6-6. Breakpoint Instruction Flowchart

Table 6-4. Exception Priority Groups

Group/ Priority	Exception and Relative Priority	Characteristic
0	0.0—Reset	Aborts all processing (instruction or exception) and does not save old context.
1	1.0—Address Error 1.1—Bus Error	Suspends processing (instruction or exception) and saves internal context.
2	2.0—BKPT, CALLM, CHK, CHK2, cp Midinstruction, cp Protocol Violation, cpTRAPcc, Divide by Zero, RTE, RTM, TRAP, TRAPcc, TRAPV	Exception processing is part of instruction execution.
3	3.0—Illegal Instruction, Line A, Unimplemented Line F, Privilege Violation, cp Preinstruction	Exception processing begins before instruction is executed.
4	4.0—cp Postinstruction 4.1—Trace 4.2—Interrupt	Exception processing begins when current instruction or previous exception processing has completed.

NOTE: 0.0 is the highest priority; 4.2 is the lowest.

The priority scheme is very important in determining the order in which exception handlers execute when several exceptions occur at the same time. As a general rule, the lower the priority of an exception, the sooner the handler routine for that exception executes. For example, if simultaneous trap, trace, and interrupt exceptions are pending, the exception processing for the trap occurs first, followed immediately by exception processing for the trace, and then for the interrupt. When the processor resumes normal instruction execution, it is in the interrupt handler, which returns to the trace handler, which returns to the trap exception handler. This rule does not apply to the reset exception; its handler is executed first even though it has the highest priority because the reset operation clears all other exceptions.

6.1.12 Return from Exception

After the MC68020/EC020 has completed exception processing for all pending exceptions, it resumes normal instruction execution at the address in the vector for the last exception processed. Once the exception handler has completed execution, the processor must return to the system context prior to the exception (if possible). The RTE instruction returns from the handler to the previous system context for any exception.

When the processor executes an RTE instruction, it examines the stack frame on top of the active supervisor stack to determine if it is a valid frame and what type of context restoration it requires. The following paragraphs describe the processing for each of the stack frame types; refer to **6.3 Coprocessor Considerations** for a description of the stack frame type formats.

For a normal four-word frame, the processor updates the SR and PC with the data read from the stack, increments the stack pointer by eight, and resumes normal instruction execution.

For the throwaway four-word frame, the processor reads the SR value from the frame, increments the active stack pointer by eight, updates the SR with the value read from the stack, and then begins RTE processing again, as shown in Figure 6-7. The processor reads a new format word from the stack frame on top of the active stack (which may or may not be the same stack used for the previous operation) and performs the proper operations corresponding to that format. In most cases, the throwaway frame is on the interrupt stack and when the SR value is read from the stack, the S and M bits are set. In that case, there is a normal four-word frame or a ten-word coprocessor midinstruction frame on the master stack. However, the second frame may be any format (even another throwaway frame) and may reside on any of the three system stacks.

For the six-word stack frame, the processor restores the SR and PC values from the stack, increments the active supervisor stack pointer by 12, and resumes normal instruction execution.

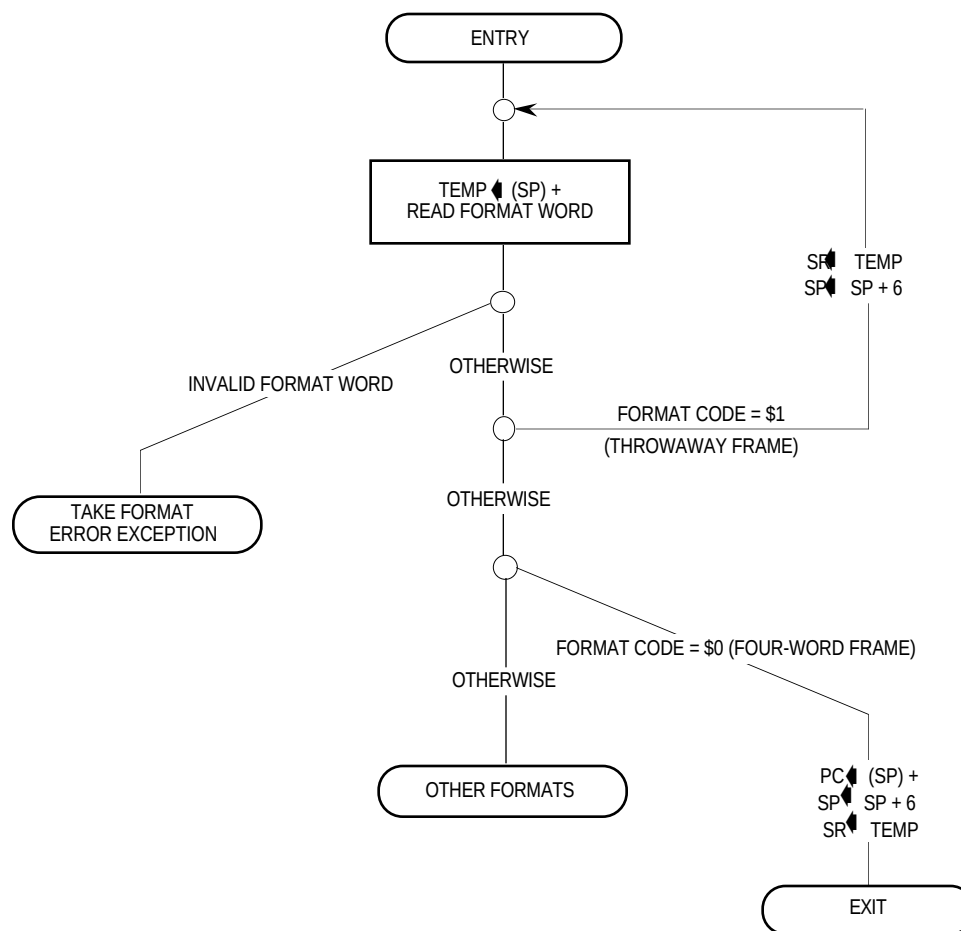


Figure 6-7. RTE Instruction for Throwaway Four-Word Frame

For the coprocessor midinstruction stack frame, the processor reads the SR, PC, instruction address, internal register values, and the evaluated effective address from the stack, restores these values to the corresponding internal registers, and increments the stack pointer by 20. The processor then reads from the response register of the coprocessor that initiated the exception to determine the next operation to be performed. Refer to **Section 7 Coprocessor Interface Description** for details of coprocessor-related exceptions.

For both the short and long bus fault stack frames, the processor first checks the format value on the stack for validity. In addition, for the long stack frame, the processor compares the version number in the stack with its own version number. The version number is located in the most significant nibble (bits 15–12) of the word at location $SP + \$36$ in the long stack frame. This validity check is required in a multiprocessor system to ensure that the data is properly interpreted by the RTE instruction. The RTE instruction also reads from both ends of the stack frame to make sure it is accessible. If the frame is invalid or inaccessible, the processor takes a format error or a bus error exception, respectively. Otherwise, the processor reads the entire frame into the proper internal registers, deallocates the stack, and resumes normal processing. Once the processor begins to load the frame to restore its internal state, the assertion of the BERR signal

causes the processor to enter the halted state. Refer to **6.2 Bus Fault Recovery** for a description of the processing that occurs after the frame is read into the internal registers.

If a format error or bus error exception occurs during the frame validation sequence of the RTE instruction, either due to any of the errors previously described or due to an illegal format code, the processor creates a normal four-word or a bus fault stack frame below the frame that it was attempting to use. In this way, the faulty stack frame remains intact. The exception handler can examine or repair the faulty frame. In a multiprocessor system, the faulty frame can be left to be used by another processor of a different type (e.g., an MC68010 or a future M68000 family processor) when appropriate.

6.2 BUS FAULT RECOVERY

An address error exception or a bus error exception indicates a bus fault. The saving of the processor state for a bus error or address error is described in **6.1.2 Bus Error Exception**, and the restoring of the processor state by an RTE instruction is described in **6.1.12 Return from Exception**.

Processor accesses of either data items or the instruction stream can result in bus errors. When a bus error exception occurs while accessing a data item, the exception is taken immediately after the bus cycle terminates. The processor may never access an instruction that is part of the instruction stream. In this case, the bus error would not be processed. For instruction faults, when the short bus fault stack frame applies, the address of the pipe stage B word is the value in the PC plus four, and the address of the stage C word is the value in the PC plus two. For the long format, the long word at SP + \$24 contains the address of the stage B word; the address of the stage C word is the address of the stage B word minus two. Address error faults occur only for instruction stream accesses, and the exceptions are taken before the bus cycles are attempted.

6.2.1 Special Status Word (SSW)

The internal SSW (see Figure 6-8) is one of several registers saved as part of the bus fault exception stack frame. Both the short bus fault format and the long bus fault format include this word at offset \$A. The bus cycle fault stack frame formats are described in detail in **6.4 Exception Stack Frame Formats**.

The SSW information indicates whether the fault was caused by an access to the instruction stream, data stream, or both. The high-order half of the SSW contains two status bits each for the B and C stages of the instruction pipe. If an address error exception occurs, the fault bits written to the stack frame are not set (they are only set due to a bus error, as previously described), and the rerun bits alone show the cause of the exception. Depending on the state of the pipeline, either RB and RC are set, or only RC is set. To correct the pipeline contents and continue execution of the suspended instruction, software must place the correct instruction stream data in the stage C and/or stage B images requested by the rerun bits and must clear the rerun bits. The least significant half of the SSW applies to data cycles only. Data and instruction stream faults may be pending simultaneously; the fault handler should be able to recognize any combination of the FC, FB, RC, RB, and DF bits.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	0
FC	FB	RC	RB	0	0	0	DF	RM	RW	SIZE	0	0	0	FC2-FC0

Figure 6-8. Special Status Word Format

FC—Fault on Stage C

When the FC bit is set, the processor attempted to use stage C and found it to be marked invalid due to a bus error on the prefetch for that stage. FC can be used by a bus error handler to determine the cause(s) of a bus error exception.

FB—Fault on Stage B

When the FB bit is set, the processor attempted to use stage B and found it to be marked invalid due to a bus error on the prefetch for that stage. FB can be used by a bus error handler to determine the cause(s) of a bus error exception.

RC—Rerun Flag for Stage C

The RC bit is set to indicate that a fault occurred during a prefetch for stage C. The RC bit is always set when the FC bit is set. The RC bit indicates that the word in stage C of the instruction pipe is invalid, and the state of the bit can be used by a handler to repair the values in the pipe after an address error or a bus error, if necessary. If the RC bit is set when the processor executes an RTE instruction, the processor may execute a bus cycle to prefetch the instruction word for stage C of the pipe (if it is required). If the RC and FC bits are set, the RTE instruction automatically reruns the prefetch cycle for stage C. The address space for the bus cycle is the program space for the privilege level indicated in the copy of the SR on the stack. If the RC bit is clear, the words on the stack for stage C of the pipe are accepted as valid; the processor assumes that there is no prefetch pending for stage C and that software has repaired or filled the image of stage C, if necessary.

1 = Rerun faulted bus cycle or run pending prefetch

0 = Do not rerun bus cycle

RB—Rerun Flag for Stage B

The RB bit is set to indicate that a fault occurred during a prefetch for stage B. The RB bit is always set when the FB bit is set. The RB bit indicates that the word in stage B of the instruction pipe is invalid, and the state of the bit can be used by a handler to repair the values in the pipe after an address error or a bus error, if necessary. If the RB bit is set when the processor executes an RTE instruction, the processor may execute a bus cycle to prefetch the instruction word for stage B of the pipe (if it is required). If the RB and FB bits are set, the RTE instruction automatically reruns the prefetch cycle for stage B. The address space for the bus cycle is the program space for the privilege level indicated in the copy of the SR on the stack. If the RB bit is clear, the words on the

stack for stage B of the pipe are accepted as valid; the processor assumes that there is no prefetch pending for stage B and that software has repaired or filled the image of stage B, if necessary.

- 1 = Rerun faulted bus cycle or run pending prefetch
- 0 = Do not rerun bus cycle

Bits 11–9—Reserved by Motorola

DF—Fault/Rerun Flag

If the DF bit is set, a data fault has occurred and caused the exception. If the DF bit is set when the processor reads the stack frame, it reruns the faulted data access; otherwise, it assumes that the data input buffer value on the stack is valid for a read or that the data has been correctly written to memory for a write (or that no data fault occurred).

- 1 = Rerun faulted bus cycle or run pending prefetch
- 0 = Do not rerun bus cycle

RM—Read-Modify-Write

- 1 = Read-modify-write operation on data cycle
- 0 = Not a read-modify-write operation

RW—Read/Write

- 1 = Read on data cycle
- 0 = Write on data cycle

SIZE—Size Code

The SIZE field indicates the size of the operand access for the data cycle.

Bit 3—Reserved by Motorola

FC2–FC0—Specifies the address space for data cycle

6.2.2 Using Software to Complete the Bus Cycles

One method of completing a faulted bus cycle is to use a software handler to emulate the cycle. This is the only method for correcting address errors. The handler should emulate the faulted bus cycle in a manner that is transparent to the instruction that caused the fault. For instruction stream faults, the handler may need to run bus cycles for both the B and C stages of the instruction pipe. The RB and RC bits of the SSW identify the stages that may require a bus cycle; the FB and FC bits of the SSW indicate that a stage was invalid when an attempt was made to use its contents. Those stages must be repaired. For each faulted stage, the software handler should copy the instruction word from the proper address space as indicated by the S-bit of the copy of the SR saved on the stack to the image of the appropriate stage in the stack frame. In addition, the handler must clear the RB or RC bit associated with the stage that it has corrected. The handler should not change the FB and FC bits.

To repair data faults (indicated by $DF = 1$), the software should first examine the RM bit in the SSW to determine if the fault was generated during a read-modify-write operation. If $RM = 0$, the handler should then check the RW bit of the SSW to determine if the fault was caused by a read or a write cycle. For data write faults, the handler must transfer the properly sized data from the data output buffer on the stack frame to the location indicated by the data fault address in the address space defined by the SSW. (Both the data output buffer and the data fault address are part of the stack frame at $SP + \$18$ and $SP + \$10$, respectively.) Data read faults only generate the long bus fault frame, and the handler must transfer properly sized data from the location indicated by the fault address and address space to the image of the data input buffer at location $SP + \$2C$ of the long format stack frame. Byte, word, and 3-byte operands are right justified in the 4-byte data buffers. In addition, the software handler must clear the DF bit of the SSW to indicate that the faulted bus cycle has been corrected.

To emulate a read-modify-write cycle, the exception handler must first read the operation word at the PC address ($SP + 2$ of the stack frame). This word identifies the CAS, CAS2, or TAS instruction that caused the fault. Then the handler must emulate this entire instruction (which may consist of up to four long-word transfers) and update the CCR portion of the SR appropriately, because the RTE instruction expects the entire operation to have been completed if the RM bit is set and the DF bit is cleared. This is true even if the fault occurred on the first read cycle.

To emulate the entire instruction, the handler must save the data and address registers for the instruction (with a MOVEM instruction, for example). Next, the handler reads and modifies (if necessary) the memory location. It clears the DF bit in the SSW of the stack frame and modifies the condition codes in the SR copy and the copies of any data or address registers required for the CAS and CAS2 instructions. Last, the handler restores the registers that it saved at the beginning of the emulation. Except for the data input buffer, the copy of the SR, and the SSW, the handler should not modify a bus fault stack frame. The only bits in the SSW that may be modified are DF, RB, and RC; all other bits, including those defined for internal use, must remain unchanged.

Address error faults must be repaired in software. Address error faults can be distinguished from bus error faults by the value in the vector offset field of the format word.

6.2.3 Completing the Bus Cycles with RTE

Another method of completing a faulted bus cycle is to allow the processor to rerun the bus cycles during execution of the RTE instruction that terminates the exception handler. This method cannot be used to recover from address errors. The RTE instruction is always executed. Unless the handler routine has corrected the error and cleared the fault (and cleared the RB/RC and DF bits of the SSW), the RTE instruction cannot complete the bus cycle(s). If the DF bit is still set at the time of the RTE execution, the faulted data cycle is rerun by the RTE instruction. If the FB or FC bit is set and the corresponding rerun bit (RB or RC) was not cleared by the software, the RTE reruns the associated instruction prefetch. The fault occurs again unless the cause of the fault, such as a nonresident page in a virtual memory system, has been corrected. If the RB or RC bit is set and the

corresponding fault bit (FB or FC) is cleared, the associated prefetch cycle may or may not be run by the RTE instruction (depending on whether the stage is required).

If a fault occurs when the RTE instruction attempts to rerun the bus cycle(s), the processor creates a new stack frame on the supervisor stack after deallocating the previous frame, and address error or bus error exception processing starts in the normal manner.

The read-modify-write operations of the MC68020/EC020 can also be completed by the RTE instruction that terminates the handler routine. The rerun operation, executed by the RTE instruction with the DF bit of the SSW set, reruns the entire instruction. If the cause of the error has been corrected, the handler does not need to emulate the instruction but can leave the DF bit set and execute the RTE instruction.

6.3 COPROCESSOR CONSIDERATIONS

Exception handler programmers should consider carefully whether to save and restore the context of a coprocessor at the beginning and end of handler routines for exceptions that can occur during the execution of a coprocessor instruction (i.e., bus errors, interrupts, and coprocessor-related exceptions). The nature of the coprocessor and the exception handler routine determines whether or not saving the state of one or more coprocessors with the cpSAVE and cpRESTORE instructions is required. If the coprocessor allows multiple coprocessor instructions to be executed concurrently, it may require its state to be saved and restored for all coprocessor-generated exceptions, regardless of whether or not the coprocessor is accessed during the handler routine. The MC68882 floating-point coprocessor is an example of this type of coprocessor. On the other hand, the MC68881 floating-point coprocessor requires FSAVE and FRESTORE instructions within an exception handler routine only if the exception handler itself uses the coprocessor.

6.4 EXCEPTION STACK FRAME FORMATS

The MC68020/EC020 provides six different stack frames for exception processing. The set of frames includes the normal four- and six-word stack frames, the four-word throwaway stack frame, the coprocessor midinstruction stack frame, and the short and long bus fault stack frames.

When the MC68020/EC020 writes or reads a stack frame, it uses long-word operand transfers wherever possible. Using a long-word-aligned stack pointer with memory that is on a 32-bit port greatly enhances exception processing performance. The processor does not necessarily read or write the stack frame data in sequential order.

The system software should not depend on a particular exception generating a particular stack frame. For compatibility with future devices, the software should be able to handle any type of stack frame for any type of exception.

Table 6-5 summarizes the stack frames defined for the MC68020/EC020.

Table 6-5. Exception Stack Frames

Stack Frames	Exception Types (Stacked PC Points to)
15 0 SP → STATUS REGISTER +\$02 PROGRAM COUNTER +\$06 0 0 0 0 VECTOR OFFSET FOUR-WORD STACK FRAME — FORMAT \$0	● Interrupt [Next instruction] ● Format Error [RTE or cpRESTORE instruction] ● TRAP #N [NEXT instruction] ● Illegal Instruction [Illegal instruction] ● A-Line Instruction [A-line instruction] ● F-Line Instruction [F-line instruction] ● Privilege Violation [First word of instruction causing Privilege Violation] ● Coprocessor Preinstruction [Opword of instruction that returned the 'take preinstruction' primitive]
15 0 SP → STATUS REGISTER +\$02 PROGRAM COUNTER +\$06 0 0 0 1 VECTOR OFFSET THROWAWAY FOUR-WORD STACK FRAME — FORMAT \$1	● Created on Interrupt Stack during interrupt exception processing when transition from master state to interrupt state occurs [Next instruction — same as on master stack]
15 0 SP → STATUS REGISTER +\$02 PROGRAM COUNTER +\$06 0 0 1 0 VECTOR OFFSET +\$08 INSTRUCTION ADDRESS SIX-WORD STACK FRAME — FORMAT \$2	● CHK [Next instruction for all these exceptions] ● CHK2 ● cpTRAPcc ● TRAPcc ● TRAPPV ● Trace ● Zero Divide ● MMU Configuration ● Coprocessor Postinstruction INSTRUCTION ADDRESS is the address of the instruction that caused the exception
15 0 SP → STATUS REGISTER +\$02 PROGRAM COUNTER +\$06 1 0 0 1 VECTOR OFFSET +\$08 INSTRUCTION ADDRESS +\$0C INTERNAL REGISTERS, 4 WORDS +\$12 COPROCESSOR MIDINSTRUCTION STACK FRAME (10 WORDS) — FORMAT \$9	● Coprocessor Midinstruction [Next word to be fetched from instruction stream for all these exceptions] ● Main-Detected Protocol Violation ● Interrupt Detected During Coprocessor Instruction (supported with 'null come again with interrupts allowed' primitive) INSTRUCTION ADDRESS is the address of the instruction that caused the exception

Table 6-5. Exception Stack Frames (Continued)

Stack Frames	Exception Types (Stacked PC Points to)
15 0 SP → STATUS REGISTER +\$02 PROGRAM COUNTER +\$06 1 0 1 0 VECTOR OFFSET +\$08 INTERNAL REGISTER +\$0A SPECIAL STATUS REGISTER +\$0C INSTRUCTION PIPE STAGE C +\$0E INSTRUCTION PIPE STAGE B +\$10 DATA CYCLE FAULT ADDRESS +\$12 INTERNAL REGISTER +\$14 INTERNAL REGISTER +\$16 INTERNAL REGISTER +\$18 DATA OUTPUT BUFFER +\$1A INTERNAL REGISTER +\$1C INTERNAL REGISTER +\$1E INTERNAL REGISTER SHORT BUS FAULT STACK FRAME (16 WORDS) — FORMAT \$A	Address Error or Bus Error — Execution Unit at Instruction Boundary [Next instruction]
15 0 SP → STATUS REGISTER +\$02 PROGRAM COUNTER +\$06 1 0 1 1 VECTOR OFFSET +\$08 INTERNAL REGISTER +\$0A SPECIAL STATUS REGISTER +\$0C INSTRUCTION PIPE STAGE C +\$0E INSTRUCTION PIPE STAGE B +\$10 DATA CYCLE FAULT ADDRESS +\$12 INTERNAL REGISTER +\$14 INTERNAL REGISTER +\$16 INTERNAL REGISTER +\$18 DATA OUTPUT BUFFER +\$1A INTERNAL REGISTER, 4 WORDS +\$22 STAGE B ADDRESS +\$24 INTERNAL REGISTERS, 2 WORDS +\$28 DATA INPUT BUFFER +\$2C INTERNAL REGISTERS, 3 WORDS +\$30 VERSION # INTERNAL INFORMATION +\$38 INTERNAL REGISTERS, 18 WORDS +\$5A LONG BUS FAULT STACK FRAME (46 WORDS) — FORMAT \$B	Address Error or Bus Error — Instruction Execution in Progress [Address of instruction in execution when fault occurred — may not be the instruction that generated the faulted bus cycle]